

FreeIPA Remote Access over Tailscale

A Cross-Border Diagnostic Case Study

Project	UK–Latvia Homelab (latvia-server)
Category	Identity Management / VPN Networking / DNS
Stack	Rocky Linux, FreeIPA, BIND (named), Tailscale (MagicDNS, Split DNS, Exit Node), firewalld, Apache (mod_auth_gssapi)

Summary

latvia-server runs FreeIPA for identity management as part of a two-site UK–Latvia homelab. The FreeIPA web UI (<https://latvia-server.lab.lan/ipa/ui/>) was reliably accessible from a PC on the local Latvian LAN, but completely unreachable from a mobile device connecting remotely over Tailscale — both on mobile data and Wi-Fi, in the UK.

What looked at first like a single connectivity fault turned out to be four independent, layered issues, each masking the next. This document walks through the full diagnostic path in the order the issues were actually found, since the order matters as much as the fixes themselves.

Architecture

- latvia-server (Rocky Linux, HP EliteDesk 800 G2 Mini) — hosts FreeIPA, BIND DNS, and Cockpit
- Local LAN: 192.168.8.14
- Tailscale mesh IP: 100.97.191.96 (MagicDNS: latvia-server.tail93370e.ts.net)
- latvia-server also serves as a Tailscale exit node for the tailnet
- Remote clients (UK-based PC and Android phone) reach the box exclusively via Tailscale when off the local LAN

Symptom

- PC on the Latvian LAN: FreeIPA loads instantly, valid TLS padlock.
- Phone in the UK (mobile data or Wi-Fi, Tailscale connected): latvia-server.lab.lan resolved, but /ipa/ui/ reliably failed with ERR_CONNECTION_TIMED_OUT.

Diagnostic Path

1. Ruled out: Tailscale split DNS

Initial hypothesis was that the phone couldn't resolve lab.lan at all. Added a Tailscale split-DNS nameserver entry (lab.lan → 100.97.191.96) via the Tailscale admin console, backed by BIND already listening correctly on the Tailscale interface. This didn't fix the issue — but confirmed resolution wasn't the (only) problem, since the hostname was resolving throughout.

2. Ruled out: firewalld zones

Checked firewall-cmd --get-active-zones and confirmed tailscale0 sat in the public zone, which already allowed https, freeipa-4, and related services. No zone misconfiguration found.

3. Ruled out: Apache binding

`ss -tulpn | grep :443` confirmed `httpd` was listening on all interfaces (`*:443`), not just the LAN IP. Not a binding issue.

4. Found: IP forwarding disabled

tailscale status surfaced a health warning — subnet routing was enabled on latvia-server, but kernel IP forwarding was not. Fixed via `sysctl`:

```
sudo sysctl -w net.ipv4.ip_forward=1
sudo sysctl -w net.ipv6.conf.all.forwarding=1
```

Made persistent via `/etc/sysctl.d/99-tailscale.conf`. Necessary for a healthy exit-node/subnet-router setup, but not the actual root cause of the FreeIPA symptom.

5. Found: packet capture showed the SYN never arriving

`tcpdump` on `tailscale0`, filtered to dst host `100.97.191.96` and port `443`, captured zero packets while reproducing the failure from the phone — even though a parallel DNS query for the same host succeeded. This ruled out TLS/Apache/Kerberos entirely: the connection attempt wasn't reaching the server at all.

Tested and ruled out the phone's "always-on exit node" setting as the cause (traffic to the exit node routing through itself) by disabling it — the fault persisted.

6. Root cause: FreeIPA's internal DNS zone answered with the LAN IP

```
dig latvia-server.lab.lan A
;; ANSWER SECTION:
latvia-server.lab.lan. 86400 IN A 192.168.8.14
```

The split-DNS fix in step 1 correctly pointed remote clients at latvia-server's BIND resolver — but that resolver's own IPA-managed zone was answering hostname queries with the local LAN address, unreachable from anywhere off the physical Latvian network. The PC had been masking this the entire time via a legacy static hosts file entry pointing directly at the Tailscale IP, added months earlier during initial FreeIPA setup.

Fix:

```
kinit admin
ipa dnsrecord-mod lab.lan latvia-server --a-rec=100.97.191.96
```

7. Secondary issue: stale local override on the PC

Removing the legacy PC-side hosts file entry (rather than leaving it in place indefinitely) surfaced a second, unrelated finding: the entry already pointed at the Tailscale IP, not the LAN IP as assumed — meaning the PC had silently depended on a manual fix for months. Commented it out once the IPA DNS record was corrected, confirmed the PC now resolves identically to the phone with no override needed.

8. Final layer: browser cache and CA trust

After the DNS fix, the phone intermittently showed a cached/offline copy of the page rather than a live connection — resolved by clearing Chrome's cached data. The final live connection attempt then surfaced `NET::ERR_CERT_AUTHORITY_INVALID` — a good sign, since it meant the TLS handshake was completing correctly for the first time; the phone simply didn't trust FreeIPA's self-signed CA.

Fix: retrieved the CA certificate from IPA's unauthenticated bootstrap endpoint:

```
http://latvia-server.lab.lan/ipa/config/ca.crt
```

(Required Tailscale to be active — this endpoint is also only reachable via the corrected DNS path.) Installed manually as a trusted CA certificate on Android via Settings → Security → Encryption & credentials → Install a certificate → CA certificate.

Outcome

<https://latvia-server.lab.lan/ipa/ui/> now loads cleanly, with a fully trusted TLS connection, from:

- PC on the local Latvian LAN
- PC in the UK, remote, over Tailscale
- Android phone in the UK, on mobile data, over Tailscale
- Android phone in the UK, on Wi-Fi, over Tailscale

All paths now resolve through the same corrected IPA DNS record — no device-specific manual overrides remain.

Root Cause, Restated

A DNS zone answering with an address that's valid on one network segment but not another is functionally identical to a wrong DNS record — it's easy to overlook specifically because it “works” from wherever it was configured, and only breaks for users on a different path. This is a common blind spot in split-network or VPN-mesh setups where a service's own DNS zone predates the remote-access layer added on top of it.

Lessons Learned / Trade-offs for Future Reference

- DNS resolving ≠ DNS resolving correctly. Every step confirmed the hostname resolved; the actual defect was in what it resolved to, not whether it resolved. dig against the authoritative server, not just a client-side lookup, was the tool that actually exposed it.
- Local workarounds hide root causes. The PC's long-standing hosts-file override prevented this bug from ever surfacing until a second device (with no such override) hit the same service.
- Packet capture beats guesswork. A single well-scoped tcpdump filter eliminated an entire category of hypotheses (TLS, Apache, Kerberos, firewalld) in one step, redirecting the investigation toward DNS and routing.
- A single-box design (exit node + DNS + IdM on one host) is efficient for a homelab but concentrates risk — noted here as a deliberate architectural trade-off rather than something to fix urgently.
- Certificate errors can be progress, not failure — ERR_CERT_AUTHORITY_INVALID after a string of connection-refused/timeout errors signaled the underlying transport was finally healthy.